Databook Client Portal: Product Brief

Oct 26, 2026 · @Someone

I want a small client portal, linked from the ACP website, where consultants publish a finished Interactive Databook (.pbix) themselves and clients view it after a secure sign-in with 2FA. It runs on Power BI Embedded ("embed for your customers") on one small Fabric F2 capacity paid by ACP. Everything after the upload is automated; I only act as support. First goal: a working demo at EUR 0 that convinces the ACP partners.

The problem

ACP builds an Interactive Databook (a Power BI report on one ACP template) for about 200 client projects a year, and wants to sell it to clients as a product, like loyos bi.

- **Sharing is manual today.** Workspaces, permissions and uploads would all go through me. I am one person and cannot do that for 200 projects.
- **Clients need a simple, secure view.** They sign in with 2FA, see only their own companies, and install nothing.
- **Why now:** the template is stable, and the partners are open to a product if a demo shows it works.

Who already does this

The pattern is common: a branded portal where each client logs in and sees only their own Power BI reports, embedded with a token and row-level security. Look at these for the look and the flow; most are vendor pages, so treat their claims as marketing.

Example	What it is	Why look at it
loyos bi	Hamburg SaaS: financial management suite on Power BI (P&L, balance sheet, cash flow from DATEV, SAP, Sage), also for tax advisors	Closest to our product idea; the partners know it
atSpark EmbedPortal	Ready-made multi-tenant portal on top of Power BI workspaces, with per-client sign-in and RLS	Shows the admin and client screens a portal needs
DataTako	White-label Power BI portal: own logo, colours, domain	Branding and client experience ideas; its RLS write-up
Star Knowledge case study	Tax and financial-planning client	Same audience as ours; clients

	portal with embedded Power BI reports	needed no Power BI licence
Power BI Embedded playground	Microsoft's live showcases of embedded reports, code open-sourced	See exactly how an embedded report looks and behaves
From report to product (Power BI Tiles blog)	Article on turning Power BI reports into a resellable client portal	Describes our business model end to end

We build our own instead of buying one of these because consultants must publish without help, and the upload checks are specific to the Databook template.

Users

Four kinds of people use the portal; only the first two use it daily.

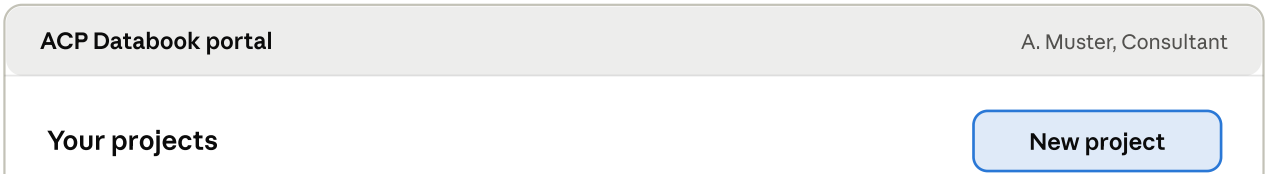
User	What they do	What they must never need
Consultant (ACP staff)	Create a project, add people, drop the .pbix, preview, release	Power BI admin knowledge, help from me
Client viewer (external)	Sign in with 2FA, view and filter the report	An install, access to other clients' data
Platform admin (me)	Watch the admin page, publish new template versions, help when a check can't be solved	Daily hands-on work per project
ACP IT	Owns the Azure subscription, Entra settings and the production deployment	Manual setup per project

Screens

Four pages carry the whole product: two for consultants, two for clients. Click through them; the layout is a suggestion, the content is what matters.

1. Consultant: projects | 2. Consultant: project page | 3. Client: sign-in | 4. Client: report

What the portal looks like (mock-up, fake data)



Project	Client companies	Status	Ends
Muster carve-out	Muster GmbH, Muster Logistik	Live	31.03.2027
Beispiel refinancing	Beispiel AG	Draft	15.01.2027
Probe vendor DD	Probe SE	Ended	30.09.2026

New project creates its own Power BI workspace automatically; ended projects lock by themselves.

Back Screen 1 of 4 Next

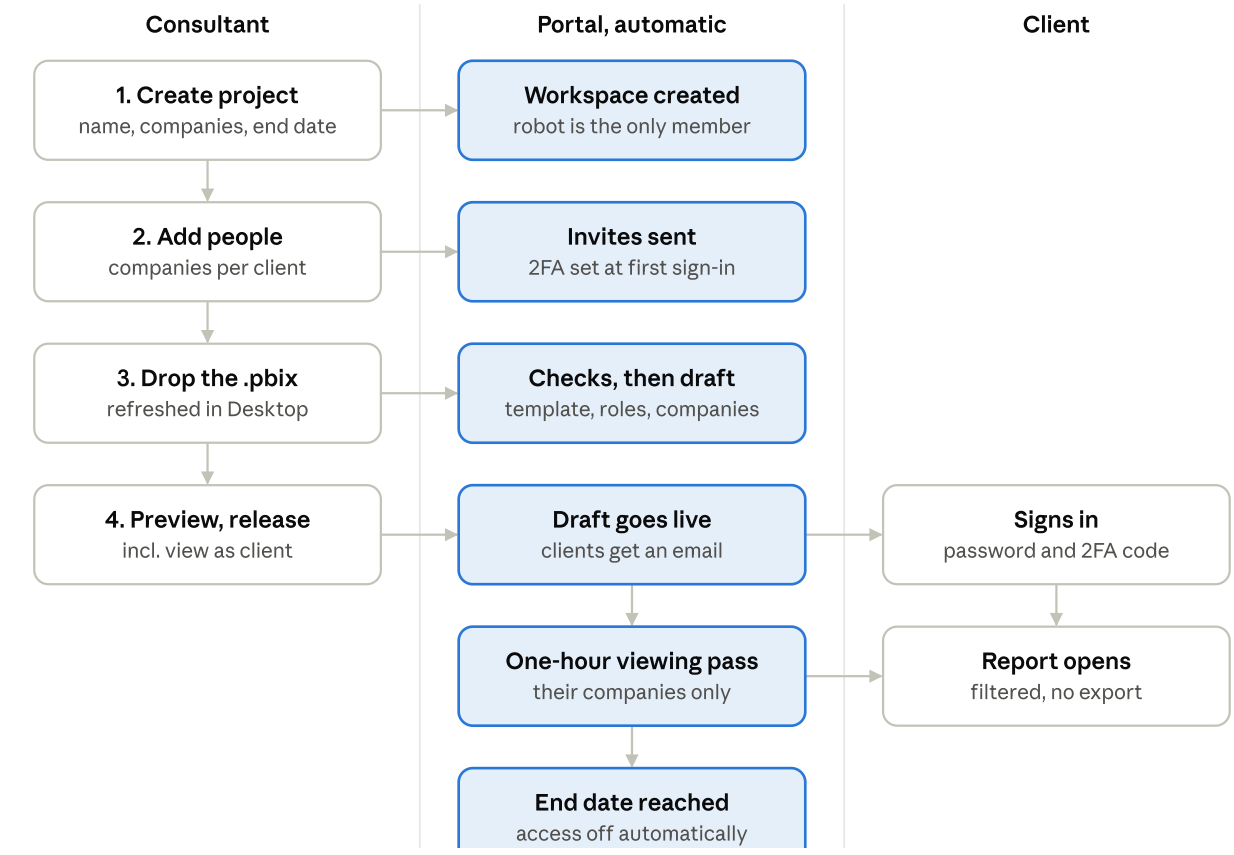
Also needed, not drawn: an admin page for me (all projects, failed uploads, usage) and an "Add person" dialog with a company picker.

How a project runs

A consultant needs about 5 minutes per project; everything shaded happens without me.

The consultant does four steps; the portal does the rest

Shaded boxes run automatically, without the admin



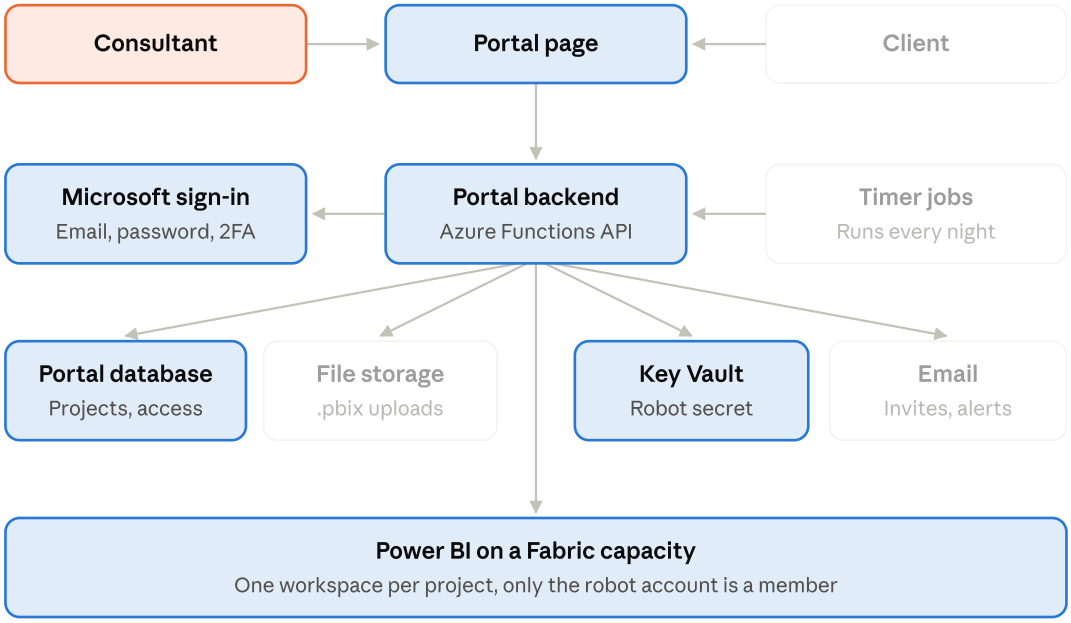
If a check in step 3 fails, the consultant gets a plain message saying what to fix (for example "This file has no Client role. Use template 1.6.") and drops the corrected file again. 14 days before the end date the project owner gets a reminder; the workspace is deleted 90 days after the end (configurable).

The backend, step by step

Only two parts are custom code: the portal page and the backend API. Everything else is an Azure or Microsoft service you configure. Click a step to see which parts act.

1. Create project | 2. Add people | 3. Drop the file | 4. Preview | 5. Release | 6. Client views | 7. Project ends | 8. Where Sejal is needed

Click a step: orange = a person acts, blue = runs automatically



Effort: About 1 minute | Sejal needed: No

The person does

- Signs in to the portal with the ACP account
- Clicks New project
- Types project name, client companies, end date

Happens automatically

- Backend creates a Power BI workspace, for example DB-1042
- Only the robot account (service principal) is added to it
- Project is saved in the portal database

Who can see it

- The consultant who created it becomes project owner
- Nobody else yet

Back Step 1 of 8 Next

Component	Suggested service	Job
Portal page	React on Azure Static Web Apps	Screens; shows the report with `powerbi-client`
Portal backend	Azure Functions (.NET or Node)	Checks sign-in and access, talks to Power BI as the robot
Microsoft sign-in	Entra ID; clients as guests	Password and 2FA; the portal never stores passwords
Portal database	Azure SQL (serverless)	Projects, people, companies per client, versions, audit log
File storage	Azure Blob Storage	Holds the .pbix until it is imported
Key Vault	Azure Key Vault	The robot account secret; never in code or the browser
Timer jobs	Azure Functions timer trigger	Reminders, expiry, workspace deletion
Email	Microsoft Graph or Azure Communication Services	Invites and release notices
Power BI	Workspaces on a Fabric capacity	One workspace per project; only the robot is a member

How to embed Power BI in the portal

The pattern is Microsoft's "embed for your customers" (App Owns Data): the backend talks to Power BI as the robot account and hands the browser a short-lived embed token; the viewer never signs in to Power BI.

Once, at setup

1. Register an app in Entra ID (this is the robot, a service principal) and store its secret in Key Vault.
2. In the Power BI admin portal, allow service principals to use the Power BI APIs, limited to one security group that holds the robot.
3. Have a Fabric capacity ready; new workspaces are assigned to it.

Per project (backend, as the robot)

Step	REST call

Create the workspace	`POST /v1.0/myorg/groups?workspaceV2=True`
Put it on the capacity	`POST /v1.0/myorg/groups/{groupId}/AssignToCapacity`
Upload the .pbix as draft	`POST /v1.0/myorg/groups/{groupId}/imports?datasetDisplayName=...`
Release: switch the live report	Store the new report and dataset ids in the portal database

Per view (every time someone opens the report)

1. The page sends the user's Entra sign-in token to `GET /api/embed/{projectId}`.
2. The backend checks the token, looks up the person's role and companies in the portal database, and calls `POST /v1.0/myorg/GenerateToken`:

```
{
  "datasets": [ { "id": "<datasetId>" } ],
  "reports":  [ { "id": "<reportId>" } ],
  "identities": [ {
    "username": "Muster GmbH|Muster Logistik GmbH",
    "roles": [ "Client" ],
    "datasets": [ "<datasetId>" ]
  } ],
  "lifetimeInMinutes": 60
}
```

For a consultant, the role is `ACP` and the username is their email. The backend returns the token, the report id and the embed URL to the page.

3. The page shows the report with `powerbi-client`:

```
import * as pbi from "powerbi-client";
const powerbi = new pbi.service.Service(pbi.factories.hpmFactory,
pbi.factories.wpmpFactory, pbi.factories.routerFactory);
const { token, reportId, embedUrl } = await fetch(`/api/embed/${projectId}`, {
headers: { Authorization: `Bearer ${idToken}` } }).then(r => r.json());
const report = powerbi.embed(document.getElementById("report"), {
  type: "report", id: reportId, embedUrl, accessToken: token,
  tokenType: pbi.models.TokenType.Embed,
  permissions: pbi.models.Permissions.Read,
  settings: { panes: { filters: { visible: false } } }
});
// before the hour is up: fetch a new token and call
report.setAccessToken(newToken)
```

The filter panel is hidden and export is switched off in the report settings, so the client can't remove the company filter or download the data.

What the Databook template must contain

The portal can only automate what the template makes checkable, so every Databook from version 1.6 on carries four fixed things. I add them to the template; the portal checks them on every upload.

Item in the .pbix	Purpose	Portal check on upload
Table `Companies` with column `Ges.`	List of legal entities in this Databook	Names match the companies entered for the project
Role `Client`	Row-level security: shows only the companies passed in the viewing pass	Role exists, otherwise reject
Role `ACP`	No filter, used for consultants	Role exists, otherwise reject
Measure `Template Version`	Tells which template the file is built on	Version 1.6 or newer, otherwise reject

The `Client` role filter is `PATHCONTAINS ( USERPRINCIPALNAME (), Companies[Ges.] )`. The backend puts the client's company list, separated by a vertical bar, into the username of the viewing pass. This must be proven in the first spike (length limit, special characters in company names).

The consultant refreshes the data in Power BI Desktop as today. The portal never refreshes and never stores data-source passwords.

Demo at EUR 0 and ground rules

The demo shows one consultant creating a project, dropping a file and releasing it, and one client signing in with 2FA and seeing only their company.

Piece	Free option for the demo
Microsoft tenant	Your own test tenant from a free Azure account
Users	1 consultant and 2 fake client users in that tenant
2FA	Entra ID security defaults (Authenticator app)
Power BI	Pro trial, 60 days; start it about a week before the

	demo
Embedding	Development embed tokens with the Pro trial, no capacity; or an F2 switched on for demo day only (about $0.36 per hour)
Portal hosting	Your laptop, or Azure Static Web Apps and Functions free tiers
Data	A Databook copy with fake "Muster" data, or a Microsoft sample report

Ground rules

- No real client data and no real Databook outside ACP's tenant. I provide a fake-data file only if the partners agree.
- Build everything as infrastructure as code (Bicep or `azd`), so ACP IT can rebuild it in ACP's tenant without changes.
- Keep secrets in Key Vault, never in the browser or in code.
- Code and documents belong to ACP; we agree that in writing before production.
- EU region only (Germany West Central or West Europe).

Test first, then build

Four short spikes decide whether the design holds; please do them before the screens.

1. **RLS through the viewing pass.** Pass two or three company names in the effective identity and check the `Client` role filters correctly, including names with spaces, umlauts and long lists.
2. **Upload by the robot account.** Import a .pbix of realistic size into a new workspace with the service principal, and read the roles and the `Template Version` measure to run the checks.
3. **F2 is big enough.** Open the heaviest Databook page (Cash Flow with all levels expanded) on F2 and watch memory and throttling. Fallback: F4 or F8.
4. **One robot or many.** Check whether service principal profiles are needed for about 200 workspaces, or whether one service principal is enough at first.

References that already exist

Start from the App-Owns-Data Starter Kit: it already has a workspace per customer, user permissions and usage tracking. It dates from 2022 (.NET 6), so use it as a blueprint and build on current .NET.

Reference	What it gives you
App-Owns-Data Starter Kit	Closest existing design: database, admin app, customer app, workspace per tenant
PowerBI-Developer-Samples	Current embed code for .NET, Node, Python with a service principal
PowerBI-JavaScript (powerbi-client)	Library that shows the report in the page
powerbi-client-react	React wrapper for the same
MSAL.js	Sign-in with Entra ID in the browser
Microsoft.Identity.Web	Sign-in and token checks in a .NET backend
Embed for your customers tutorial	Step-by-step first embed
Row-level security with embedding	How the effective identity and roles work
Embed with a service principal	Setting up the robot account
Multitenancy guide	Organising workspaces for many customers
Service principal profiles and Create Profile API	Scaling past one robot account
Power BI Dev Camp links	Videos and samples behind the Starter Kit
Power BI Embedded playground	Try embed features without code
Microsoft Fabric pricing	Current F SKU prices per region
BMAD Method	Optional way of working with AI coding agents; Patrik used it for his product

ACP material

- Partner page: why a Databook client portal
- Detailed requirements (draft, paused), with 24 functional requirements and the portal data model